

4. PENGUJIAN

Bab ini akan membahas pengujian-pengujian yang dilakukan pada sistem untuk melihat apakah sistem sudah berjalan sebagaimana mestinya dan telah menjawab masalah-masalah yang dirumuskan. Beberapa proses pengujian yang dilakukan adalah pengujian skenario, *usability testing*, *User Acceptance Testing* (UAT), dan pengujian efisiensi sistem.

4.1. Pengujian berdasarkan skenario

Pengujian skenario akan menguji sistem berdasarkan beberapa skenario bisnis yang mungkin terjadi pada Toko X untuk melihat apakah sistem dapat menghasilkan perilaku yang diinginkan dari masing-masing skenario. Skenario-skenario ini meliputi beberapa kegiatan inti dalam manajemen stok seperti manajemen produk, pencatatan pembelian dan penjualan, serta *stock opname*. Skenario diasumsikan dilakukan oleh pemilik toko sebagai admin.

4.1.1. Skenario membuat produk baru

Pemilik toko ingin menambahkan sebuah produk baru yang akan dijual di tokonya. Produk termasuk ke jenis popok dan bernama “popok sella”. *Input* data produk dapat dilihat pada gambar 4.1.

ID	Nama
1	popok lyla
2	popok dio
3	popok filla
4	popok pio
5	grito bayi
6	baju putih
7	baju warna
8	celana pendek

Nama Varian	Harga Standar	Jumlah Stok
standar	2500	0

Gambar 4.1 *Screenshot* tampilan *input* data produk baru

Produk yang baru saja ditambahkan akan masuk ke tabel “produk” dan “produk_varian” di dalam *database*. Penambahan data pada *database* dapat dilihat pada gambar 4.2 untuk tabel “produk” dan gambar 4.3 untuk tabel “produk_varian”.

```

1 SELECT * FROM produk
2 WHERE nama='Popok sella';

```

id	nama	id_subkategor	id_supplier	qty_minimum
63	Popok sella	5	2	12

Gambar 4.2 Screenshot hasil data produk pada *database*

```

1 SELECT * FROM produk_varian v
2 JOIN produk p ON p.id = v.id_produk
3 WHERE p.nama='Popok sella';

```

id	varian	id_produk	harga_standar	jumlah_stok	stok_reservas	nama	id_subkategor	id_supplier	qty_m
63	Standar	63	2500	0	0	Popok sella	5	2	12

Gambar 4.3 Screenshot hasil data varian produk pada *database*

4.1.2. Skenario mencatat proses pesanan pembelian

Pada skenario ini, pemilik toko ingin membeli produk “popok sella” sebanyak 60 buah (5 lusin), dimana saat ini jumlah stok untuk produk tersebut adalah 0. Pemilik toko melakukan pemesanan kepada *supplier* dan kemudian mencatat pesanan tersebut dengan menambahkan pesanan pembelian baru pada sistem. *Input* pesanan pembelian dapat dilihat pada gambar 4.4.

Tambah Pesanan Pembelian draft

Tanggal Pemesanan: 05/30/2025 11:32

Supplier: Dio (Contact Person: Anita)

Produk:

Produk	Varian	Qty. Dipesan
popok sella	standar	60

+ Tambah Produk

Tutup Simpan

Gambar 4.4 Screenshot tampilan *input* pesanan pembelian baru

Penambahan pesanan baru tersebut akan masuk ke tabel “pesanan_pembelian” dan “item_pesanan_pembelian” pada *database*, dimana dapat dilihat pada gambar 4.5.

```
1 SELECT po.id AS "ID PO", po.status_pesanan, po.tanggal_pesana, p.nama AS produk, v.varian, i.qty_dipesan
2 FROM item_pesanan_pembelian i
3 JOIN pesanan_pembelian po ON po.id = i.id_beli
4 JOIN produk_varian v ON v.id = i.id_varian
5 JOIN produk p ON p.id = v.id_produk
6 WHERE nama='popok sella';
7
8
```

Results Chart Export

Source Primary Database Role postgres Run CTRL ↵

ID PO	status_pesana	tanggal_pesana	produk	varian	qty_dipesan
15	dipesan	2025-05-30 04:32:00+00	popok sella	standar	60

Gambar 4.5 *Screenshot* hasil data pesanan pembelian pada *database*

Tiga hari kemudian, pesanan dikirim ke gudang dan diterima oleh pemilik toko. Setelah memeriksa kiriman barang, pemilik toko akan meng-*update* data pesanan dengan memencet tombol *edit* dan mengubah status ke “diterima” dari “dipesan”. *Form* pesanan akan berubah dan pemilik toko mengisi data terkait penerimaan barang seperti dapat dilihat pada gambar 4.6. Hasil perubahan ini akan memperbarui data pesanan pada *database* seperti yang dapat dilihat pada gambar 4.7.

Pembelian

#15

Tanggal Pesanan
Supplier: Dio (Contact Person: Anita)

Tanggal Diterima
Lokasi Penerimaan
No. Surat Jalan

Edit Pesanan Pembelian

dipesanditerimadibayar

X

Tanggal Pemesanan
05/30/2025 11:32

Supplier
Dio (Contact Person: Anita)

Produk

Produk	Varian	Qty. Dipesan	Qty. Diterima	Qty. Rusak	Harga Beli (satuan)
popo	sta	60	60	0	2375

Tanggal Diterima
06/02/2025 09:42

Lokasi Penerimaan
Gudang

No. Surat Jalan
D202505-207

Batalan

TutupUpdate

Gambar 4.6 Screenshot tampilan *update* pesanan pembelian ke “diterima”

```
1 SELECT po.id AS "ID PO", po.status_pesanan, po.tanggal_diterima, po.lokasi_penerimaan, p.nama AS produk, v.varian, i.
   qty_diterima, i.qty_rusak
2 FROM item_pesanan_pembelian i
3 JOIN pesanan_pembelian po ON po.id = i.id_beli
4 JOIN produk_varian v ON v.id = i.id_varian
5 JOIN produk p ON p.id = v.id_produk
6 WHERE nama='popok sella';
7
```

Results Chart Export
Source Primary Database Role postgres Run CTRL ⌘

ID PO	status_pesana	tanggal_diterima	lokasi_peneri	produk	varian	qty_diterima	qty_rusak
15	diterima	2025-06-02 02:42:00+00	gudang	popok sella	standar	60	0

Gambar 4.7 Screenshot hasil perubahan data pesanan pembelian pada status “diterima”

Sistem juga akan secara otomatis menambahkan jumlah stok dari produk dengan kuantitas barang yang diterima secara utuh karena tidak ada barang rusak yang rusak di dalam skenario ini. Perubahan data stok pada database dapat dilihat pada gambar 4.8.

```

38 SELECT v.jumlah_stok
39 FROM produk_varian v
40 JOIN produk p ON p.id = v.id_produk
41 WHERE p.nama = 'popok sella';
42
43
44
45

```

jumlah_stok
60

Gambar 4.8 Screenshot hasil perubahan data jumlah stok dari item pesanan

Selanjutnya, karena telah melakukan pembayaran untuk pesanan tersebut, pemilik toko kembali meng-*update* data pesanan. Pemilik toko dapat mengikuti cara yang sama seperti sebelumnya dan mengubah status menjadi “dibayar” dari “diterima”. *Form* akan menyesuaikan dengan status “dibayar” seperti dapat dilihat pada gambar 4.9. Penambahan data pembayaran juga akan tercatat di *database* dan dapat dilihat pada gambar 4.10.

Edit Pesanan Pembelian [dipesan] [diterima] [dibayar]

Tanggal Pesanan: 05/30/2025 11:32 Supplier: Dio (Contact Person: Anita)

Produk

Produk	Varian	Qty. Dipesan	Qty. Diterima	Qty. Rusak	Harga Beli (satuan)
popo	sta	60	60	0	2375

Tanggal Pembayaran: 05/30/2025 09:44 Total Dibayarkan: 142500 Alat Pembayaran: Tunai

[Selesaikan] [Tutup] [Update]

Gambar 4.9 Screenshot tampilan *update* pesanan pembelian ke “dibayar”

```

1 SELECT po.id AS "ID PO", po.status_pesanan, po.tanggal_pembayaran, po.total_dibayarkan, po.alat_pembayaran, p.nama AS
   produk, v.varian
2 FROM item_pesanan_pembelian i
3 JOIN pesanan_pembelian po ON po.id = i.id_beli
4 JOIN produk_varian v ON v.id = i.id_varian
5 JOIN produk p ON p.id = v.id_produk
6 WHERE nama='popok sella';
7

```

ID PO	status_pesana	tanggal_pembayaran	total_dibayar	alat_pembayar	produk	varian
15	dibayar	2025-05-30 02:44:00+00	142500	tunai	popok sella	standar

Gambar 4.10 *Screenshot* hasil perubahan data pesanan pembelian pada status “dibayar”

Setelah pembayaran selesai, pemilik toko ingin menyelesaikan pesanan dengan menekan tombol “selesaikan” pada kartu pesanan. Tampilan tombol dapat dilihat pada gambar 4.11.

#15 dibayar

Tanggal Pemesanan: 30/05/2025, 11.32
Supplier: Dio (Contact Person: Anita)
Items:
popok sella - standar 60

Selesaikan

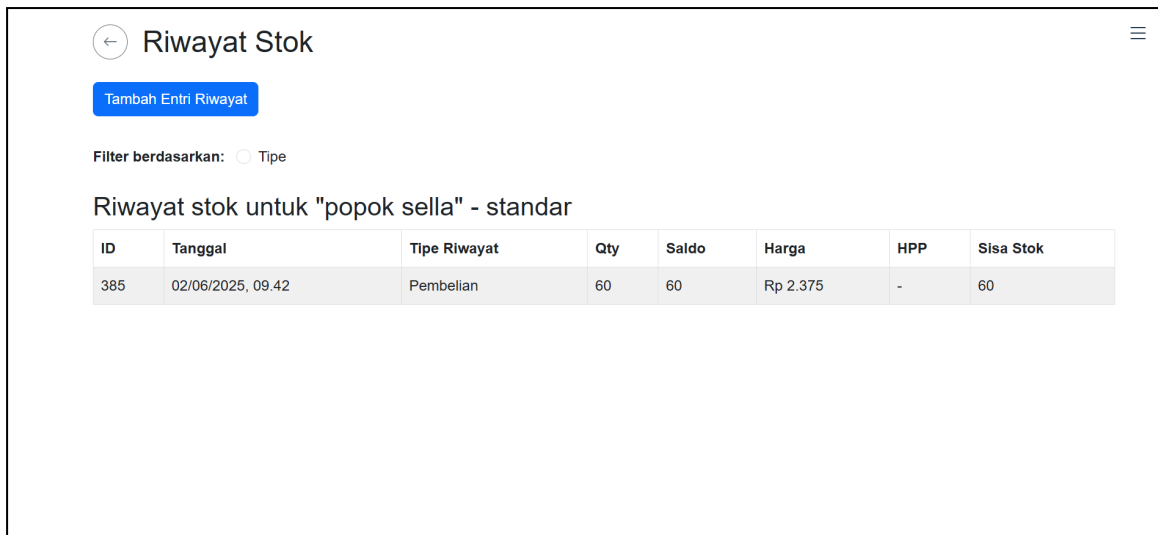
Gambar 4.11 *Screenshot* tampilan tombol penyelesaian pada kartu pesanan pembelian

Aksi ini akan mengubah status pesanan menjadi selesai seperti yang dapat dilihat pada gambar 4.12. Selain itu, data perubahan stok juga akan dicatat sebagai entri pembelian baru di riwayat stok item di dalam pesanan. Tampilan riwayat stok dapat dilihat pada gambar 4.13 dan hasil penambahan data pada *database* dapat dilihat pada gambar 4.14.

```
1 SELECT po.id AS "ID PO", po.status_pesanan, p.nama AS produk, v.varian
2 FROM item_pesanan_pembelian i
3 JOIN pesanan_pembelian po ON po.id = i.id_beli
4 JOIN produk_varian v ON v.id = i.id_varian
5 JOIN produk p ON p.id = v.id_produk
6 WHERE nama='popok sella';
7
8
```

ID PO	status_pesanan	produk	varian
15	selesai	popok sella	standar

Gambar 4.12 *Screenshot* hasil perubahan data pesanan pembelian setelah diselesaikan



Gambar 4.13 *Screenshot* tampilan data riwayat pembelian pada *website*



Gambar 4.14 *Screenshot* hasil data penambahan entri riwayat pembelian

4.1.2.1. Skenario barang rusak

Seminggu kemudian, pemilik toko ingin kembali membeli produk “popok sella”. Proses pencatatan dijalankan seperti sebelumnya. Tetapi saat barang diterima, ditemukan bahwa ada 3 buah barang yang rusak sehingga tidak layak untuk dijual. Meskipun tidak ada sistem retur, pemilik toko perlu mencatat kerusakan ini untuk menjaga konsistensi data jumlah stok. Input pencatatan barang rusak dapat dilihat pada gambar 4.15.

Gambar 4.15 Screenshot tampilan input kuantitas barang rusak

Adanya data kerusakan ini, akan membuat sistem untuk menyesuaikan jumlah stok yang ditambahkan. Penambahan jumlah stok dari kuantitas barang yang diterima akan dikurangi dengan kuantitas barang yang rusak. Pada skenario ini, jumlah stok produk “popok sella” awalnya sejumlah 48 buah sehingga jika ditambahkan 24 buah dari barang yang diterima dan dikurangi 3 buah dari barang yang rusak maka akan menjadi 69 buah. Data stok pada database dapat dilihat pada gambar 4.16.

```

1 SELECT po.id AS "ID PO", po.status_pesanan, p.nama AS produk, v.varian, i.qty_diterima, i.qty_rusak, v.jumlah_stok
2 FROM item_pesanan_pembelian i
3 JOIN pesanan_pembelian po ON po.id = i.id_beli
4 JOIN produk_varian v ON v.id = i.id_varian
5 JOIN produk p ON p.id = v.id_produk
6 WHERE po.id=16;
7

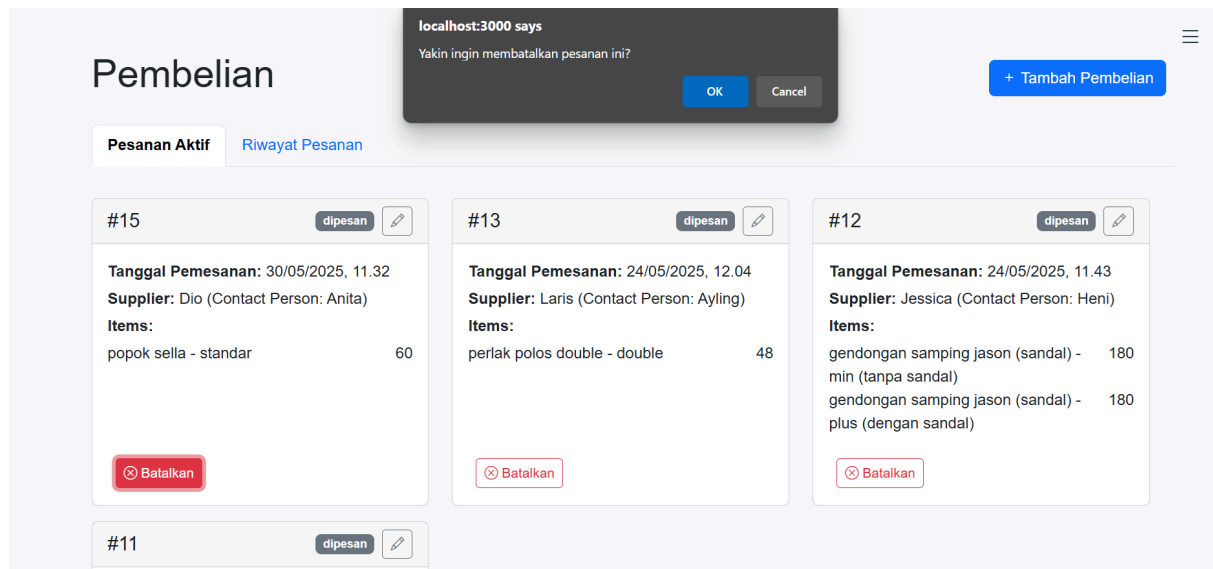
```

ID PO	status_pesana	produk	varian	qty_diterima	qty_rusak	jumlah_stok
16	diterima	popok sella	standar	24	3	69

Gambar 4.16 Screenshot hasil data jumlah stok dengan barang rusak

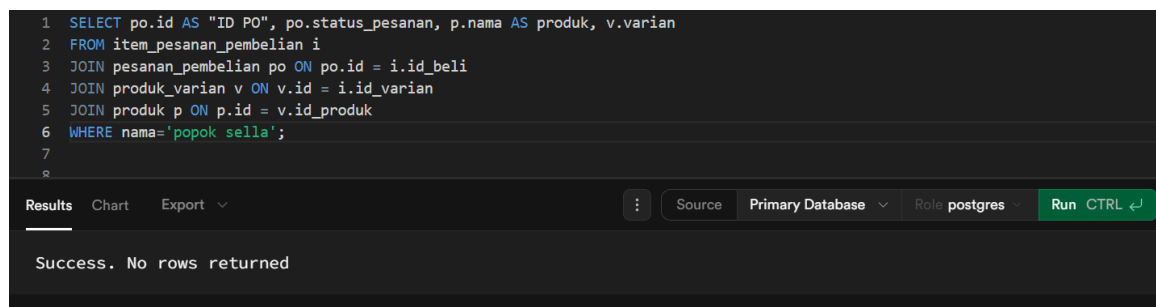
4.1.2.2. Skenario membatalkan pesanan

Terdapat pula skenario dimana pesanan tidak berlanjut dari status “dipesan”. Apabila status pesanan masih “dipesan”, akan terdapat tombol *cancel* pada kartu pesanan. Pemilik toko melakukan pesanan pembelian untuk produk “popok sella” dan sudah mencatat data pesannya. Tetapi pemilik toko berubah pikiran dan ingin membatalkan pesanan yang telah dibuat. Maka pemilik toko menekan tombol *cancel* pada pesanan yang diinginkan untuk menghapusnya. Tampilan pembatalan dapat dilihat pada gambar 4.17.



Gambar 4.17 Screenshot tampilan peringatan membatalkan pesanan pembelian

Kemudian, sistem akan menghapus data terkait dari *database* seperti yang dapat dilihat pada gambar 4.18. *Query* berhasil dijalankan tetapi tidak ada baris data yang dikeluarkan, dimana artinya data terkait berhasil dihapus.



Gambar 4.18 Screenshot hasil penghapusan data pesanan pembelian

4.1.3. Skenario mencatat proses pesanan penjualan

Pada skenario ini, seorang bakul dari luar kota ingin membeli produk “popok sella” sebanyak 24 buah (2 lusin) yang akan diambil keesokan harinya oleh seorang perwakilan. Pemilik toko kemudian menambahkan data penjualan baru untuk mencatat pesanan tersebut. *Input* pesanan penjualan dapat dilihat pada gambar 4.19.

Gambar 4.19 Screenshot tampilan *input* pesanan penjualan baru

Penambahan pesanan baru tersebut akan masuk ke tabel “pesanan_penjualan” dan “item_pesanan_penjualan” pada *database* yang dapat dilihat pada gambar 4.20 serta menambahkan jumlah stok reservasi pada produk yang dipesan dan dapat dilihat pada gambar 4.21.

```

15 SELECT pp.id AS "ID Penjualan", pp.status_pesanan, pp.status_persiapan, pp.tanggal_pesan, p.nama AS produk, v.varian, i.
16   qty_dipesan
17 FROM item_pesanan_penjualan i
18 JOIN pesanan_penjualan pp ON pp.id = i.id_jual
19 JOIN produk_varian v ON v.id = i.id_varian
20 JOIN produk p ON p.id = v.id_produk
21 WHERE nama='popok sella';

```

ID Penjualan	status_pesana	status_persia	tanggal_pesan	produk	varian	qty_dipesan
108	dipesan	false	2025-06-03 04:34:00+00	popok sella	standar	24

Gambar 4.20 Screenshot hasil data pesanan penjualan pada *database*

```

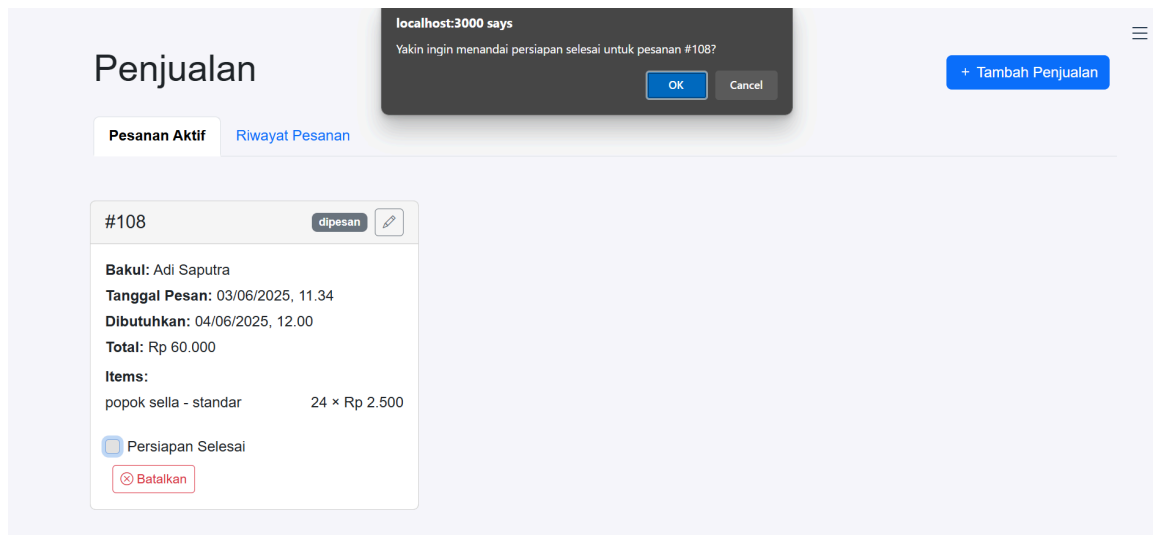
23 SELECT v.stok_reservasi
24 FROM produk_varian v
25 JOIN produk p ON p.id = v.id_produk
26 WHERE p.nama = 'popok sella';
27
28
29

```

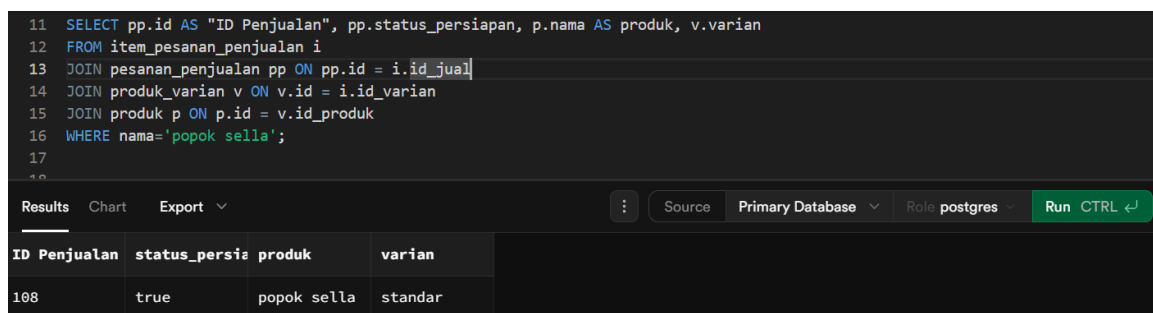
stok_reservas
24

Gambar 4.21 Screenshot perubahan data jumlah stok reservasi

Pemilik toko kemudian mempersiapkan produk yang dipesan oleh bakul. Ketika sudah selesai, pemilik toko mencentang *checkbox* untuk menandakan persiapan selesai. Tampilannya pada *website* dapat dilihat pada gambar 4.22 dan perubahan data status persiapan dapat dilihat pada gambar 4.23.



Gambar 4.22 Screenshot tampilan konfirmasi persiapan pesanan penjualan



Gambar 4.23 Screenshot hasil perubahan data status persiapan

Keesokan harinya, perwakilan dari bakul datang untuk mengambil barang yang sudah dipesan sebelumnya. Pemilik toko meng-*update* data pesanan dengan memencet tombol *edit* dan mengubah status ke “diambil” dari “dipesan”. *Form* pesanan akan berubah dan pemilik toko mengisi data terkait pengambilan barang seperti dapat dilihat pada gambar 4.24. Hasil perubahan ini akan memperbarui data pesanan pada *database* seperti yang terlihat pada gambar 4.25.

The screenshot shows a web application interface for managing sales orders. A modal window titled "Edit Pesanan Penjualan" is open, displaying details for order #108. The status is updated from "dipesan" to "diambil".

Form Fields:

- Tanggal Pesan:** 06/03/2025 11:34
- Tanggal Butuh:** 06/04/2025 12:00
- Bakul:** Adi Saputra
- Produk Table:**

Produk	Varian	Qty. Dipesan	Harga	Subtotal
popok sella	standar	24	2500	Rp 60.000
- Total:** Rp 60.000
- Tanggal Diambil:** 06/04/2025 12:02
- Pihak Pengambil:** Perwakilan

Buttons: Batalakan, Tutup, Update

Gambar 4.24 Screenshot tampilan *update* pesanan penjualan ke “diambil”

The screenshot shows a SQL query result in a database client. The query selects order details for the product 'popok sella'.

```

22 SELECT pp.id AS "ID Penjualan", pp.status_pesanan, pp.tanggal_diambil, pp.pihak_pengambil, p.nama AS produk, v.varian, i.
   qty_dipesan
23 FROM item_pesanan_penjualan i
24 JOIN pesanan_penjualan pp ON pp.id = i.id_jual
25 JOIN produk_varian v ON v.id = i.id_varian
26 JOIN produk p ON p.id = v.id_produk
27 WHERE nama='popok sella';

```

Results Table:

ID Penjualan	status_pesanan	tanggal_diambil	pihak_pengambil	produk	varian	qty_dipesan
108	diambil	2025-06-04 05:02:00+00	perwakilan	popok sella	standar	24

Gambar 4.25 Screenshot hasil perubahan data pesanan pada status “diambil”

Pada skenario ini, bakul melakukan pembayaran pada keesokan paginya ketika barang sudah diterima dan diperiksa. Pemilik toko mengkonfirmasi pembayaran dan, dengan mengikuti cara yang sama seperti sebelumnya, mengubah status menjadi “dibayar” dari “diambil”. *Form* akan menyesuaikan dengan status “dibayar” seperti yang dapat dilihat pada gambar 4.26.

The screenshot shows the same "Edit Pesanan Penjualan" modal window, but the status is now updated to "dibayar".

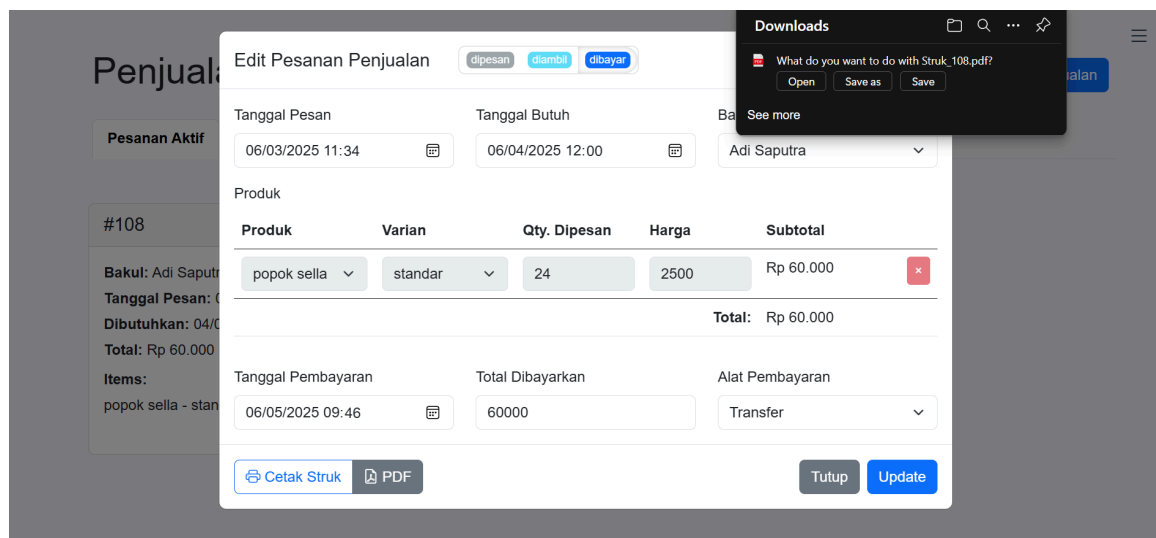
Form Fields:

- Tanggal Pesan:** 06/03/2025 11:34
- Tanggal Butuh:** 06/04/2025 12:00
- Bakul:** Adi Saputra
- Produk Table:** (Same as in Gambar 4.24)
- Total:** Rp 60.000
- Tanggal Pembayaran:** 06/05/2025 09:46
- Total Dibayarkan:** 60000
- Alat Pembayaran:** Transfer

Buttons: Cetak Struk, PDF, Tutup, Update

Gambar 4.26 Screenshot tampilan *update* pesanan penjualan ke “dibayar”

Pemilik toko pun mencetak struk pembelian dalam bentuk PDF dan mengirimkannya kepada bakul. Tampilan pengunduhan struk dapat dilihat pada gambar 4.27. Sedangkan, isi PDF struk dapat dilihat pada gambar 4.28.



Gambar 4.27 *Screenshot* tampilan pengunduhan struk pesanan



Gambar 4.28 *Screenshot* tampilan isi PDF struk pesanan

Penambahan data pembayaran ini akan tercatat di *database* seperti yang dapat dilihat pada gambar 4.29. Setelah pembayaran selesai, pemilik toko ingin menyelesaikan pesanan dengan menekan tombol “selesaikan” pada kartu pesanan, dimana tampilannya dapat dilihat pada

gambar 4.30. Aksi penyelesaian pesanan ini akan mengubah status pesanan selesai seperti dapat dilihat datanya pada gambar 4.31.

```

22 SELECT pp.id AS "ID Penjualan", pp.status_pesanan, pp.tanggal_dibayar, pp.total_dibayarkan, pp.alat_pembayaran, p.nama AS
23 produk, v.varian
24 FROM item_pesanan_penjualan i
25 JOIN pesanan_penjualan pp ON pp.id = i.id_jual
26 JOIN produk_varian v ON v.id = i.id_varian
27 JOIN produk p ON p.id = v.id_produk
28 WHERE nama='popok sella';
29

```

ID Penjualan	status_pesana	tanggal_dibayar	total_dibayar	alat_pembayar	produk	varian
108	dibayar	2025-06-05 02:46:00+00	60000	transfer	popok sella	standar

Gambar 4.29 Screenshot hasil perubahan data pesanan penjualan pada status “dibayar”

#108

dibayar

Bakul: Adi Saputra

Tanggal Pesan: 03/06/2025, 11.34

Dibutuhkan: 04/06/2025, 12.00

Total: Rp 60.000

Items:

popok sella - standar 24 × Rp 2.500

Selesaikan

Cetak

Gambar 4.30 Screenshot tampilan tombol penyelesaian pada kartu pesanan penjualan

```

22 SELECT pp.id AS "ID Penjualan", pp.status_pesanan, p.nama AS produk, v.varian
23 FROM item_pesanan_penjualan i
24 JOIN pesanan_penjualan pp ON pp.id = i.id_jual
25 JOIN produk_varian v ON v.id = i.id_varian
26 JOIN produk p ON p.id = v.id_produk
27 WHERE nama='popok sella';
28
29

```

ID Penjualan	status_pesana	produk	varian
108	selesai	popok sella	standar

Gambar 4.31 Screenshot hasil perubahan data pesanan penjualan setelah diselesaikan

Penyelesaian pesanan tersebut akan menyesuaikan data jumlah stok serta stok reservasi seperti yang dapat dilihat pada gambar 4.32. Selain itu, data perubahan stok akan dicatat juga pada riwayat stok sebagai entri penjualan. Tampilan riwayat stok dapat dilihat pada gambar 4.33 dan hasil penambahan data pada database dapat dilihat pada gambar 4.34.

```

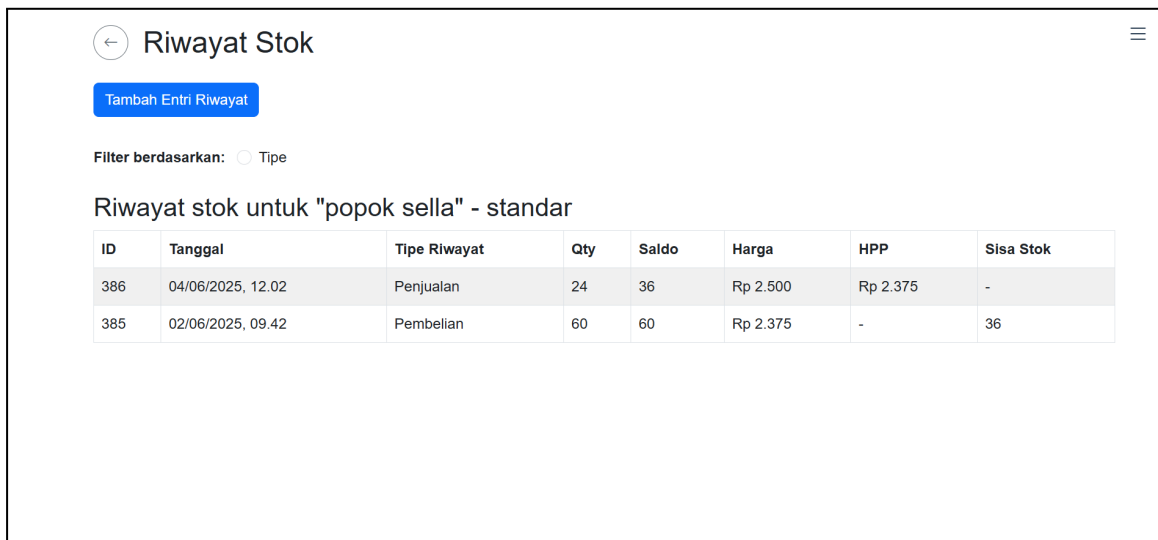
38 SELECT v.jumlah_stok, v.stok_reservasi
39 FROM produk_varian v
40 JOIN produk p ON p.id = v.id_produk
41 WHERE p.nama = 'popok sella';
42
43
44
45

```

Results Chart Export ✓ Source Primary Database Role postgres Run CTRL ↵

jumlah_stok	stok_reservas
36	0

Gambar 4.32 Screenshot hasil penyesuaian data stok setelah pesanan selesai



Gambar 4.33 Screenshot tampilan data riwayat penjualan pada website

```

46 SELECT h.id, h.tanggal, h.tipe_riwayat, p.nama || '-' || v.varian AS "produk", h.qty, h.saldo, h.harga, h.hpp, h.id_referensi, h.stok_sisa
47 FROM riwayat_stok h
48 JOIN produk_varian v ON v.id = h.id_varian
49 JOIN produk p ON p.id = v.id_produk
50 WHERE nama='popok sella'
51 ORDER BY tanggal DESC;

```

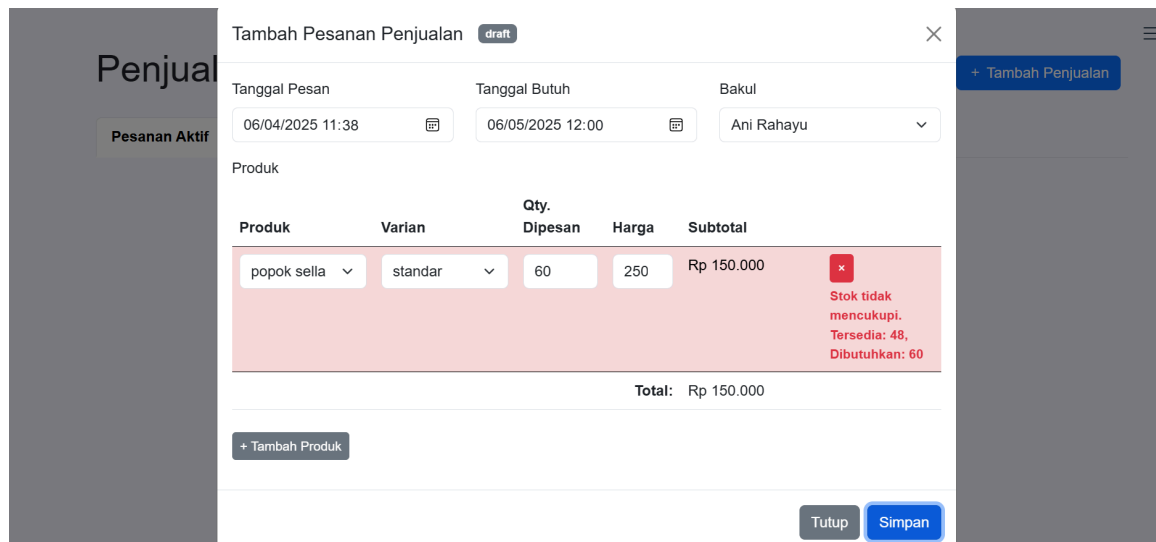
Results Chart Export Source Primary Database Role postgres Run CTRL ↵

id	tanggal	tipe_riwayat	produk	qty	saldo	harga	hpp	id_referensi	stok_sisa
386	2025-06-04 05:02:00+00	penjualan	popok sella-standar	24	36	2500	2375	108	NULL
385	2025-06-02 02:42:00+00	pembelian	popok sella-standar	60	60	2375	NULL	15	36

Gambar 4.34 Screenshot hasil data penambahan entri riwayat penjualan

4.1.3.1. Skenario jumlah stok tidak mencukupi

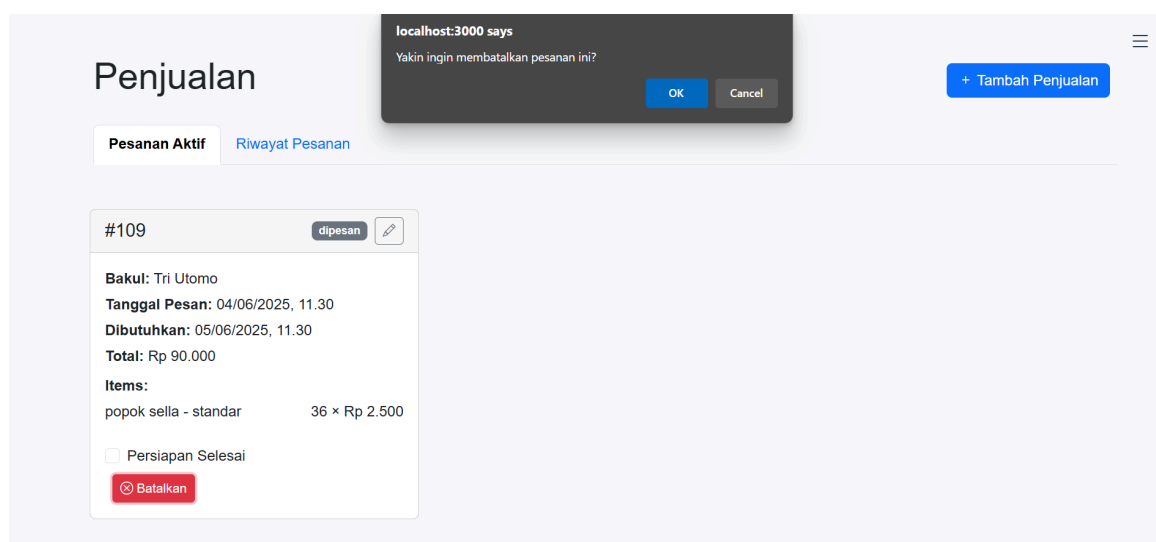
Ada seorang bakul lain yang ingin kembali membeli produk “popok sella” sebanyak 60 buah (5 lusin) sedangkan stok yang dimiliki hanya 48 buah. Pemilik toko lupa mengecek data jumlah stok sebelumnya dan mulai mencatat pesanan baru pada sistem. Sistem tidak akan membuat pesanan ini dan memberi peringatan kepada pemilik toko bahwa stok tidak mencukupi. Tampilan dari skenario ini dapat dilihat pada gambar 4.35.



Gambar 4.35 Screenshot tampilan peringatan stok tidak mencukupi

4.1.3.2. Skenario membatalkan pesanan

Terdapat pula skenario dimana pesanan tidak berlanjut dari status “dipesan”. Apabila status pesanan masih “dipesan”, akan terdapat tombol *cancel* pada kartu pesanan. Seorang bakul melakukan pemesanan produk “popok sella” dan pemilik toko sudah mencatat pesanan ini. Tetapi kemudian, bakul membatalkan pesannya maka pemilik toko ingin menghapus data pesanan yang telah dibuat. Pemilik toko menekan tombol *cancel* pada kartu pesanan tersebut seperti pada gambar 4.36 dan sistem menghapus data terkait dari *database* yang hasilnya dapat dilihat pada gambar 4.37.



Gambar 4.36 Screenshot tampilan peringatan membatalkan pesanan penjualan

```

9 SELECT pp.id AS "ID Penjualan", pp.status_persiapan, p.nama AS produk, v.varian
10 FROM item_pesanan_penjualan i
11 JOIN pesanan_penjualan pp ON pp.id = i.id_jual
12 JOIN produk_varian v ON v.id = i.id_varian
13 JOIN produk p ON p.id = v.id_produk
14 WHERE nama='popok sella';
15
16

```

Results Chart Export ▾

Source Primary Database ▾ Role postgres ▾ Run CTRL ↵

Success. No rows returned

Gambar 4.37 Screenshot hasil penghapusan data pesanan penjualan

Penambahan jumlah pada stok reservasi yang sebelumnya dilakukan juga akan dikembalikan ke semula. Data stok reservasi sebelum dan sesudah pembatalan dapat dilihat pada gambar 4.38 dan gambar 4.39.

```

27 SELECT v.stok_reservasi
28 FROM produk_varian v
29 JOIN produk p ON p.id = v.id_produk
30 WHERE p.nama = 'popok sella';
31
32
33
34

```

Results Chart Export ▾

Source Primary Database ▾ Role postgres ▾ Run CTRL ↵

stok_reservas
36

Gambar 4.38 Screenshot tampilan data jumlah stok reservasi sebelum pembatalan

```

27 SELECT v.stok_reservasi
28 FROM produk_varian v
29 JOIN produk p ON p.id = v.id_produk
30 WHERE p.nama = 'popok sella';
31
32
33
34

```

Results Chart Export ▾

Source Primary Database ▾ Role postgres ▾ Run CTRL ↵

stok_reservas
0

Gambar 4.39 Screenshot tampilan data jumlah stok reservasi setelah pembatalan

4.1.4. Skenario menambahkan data *supplier*

Pemilik toko memiliki *supplier* baru yang akan menjadi tempat langganan pembelian. *Supplier* merupakan sebuah perusahaan bernama “Abadi Jaya” dengan *contact person* bernama Ali. *Supplier* ini berasal dari kota Bandung. Untuk memudahkan manajemen *supplier* kedepannya, pemilik toko ingin menambahkan data *supplier* baru ke dalam sistem. *Input* data *supplier* baru dapat dilihat pada gambar 4.40. Kemudian hasil penambahan data pada tabel *supplier* di dalam database dapat dilihat pada gambar 4.41.

Tambah Supplier

Perusahaan
Abadi Jaya

Contact Person
Ali

No. HP
081122334455

Kota
Bandung

Tutup Tambah

ID	Perusahaan
1	Areta Baby
2	Dio
3	Laris
4	-
5	-
6	-
7	Larissa

Gambar 4.40 Screenshot tampilan *input* data *supplier* baru

```

53 SELECT * FROM supplier
54 WHERE perusahaan = 'Abadi Jaya';

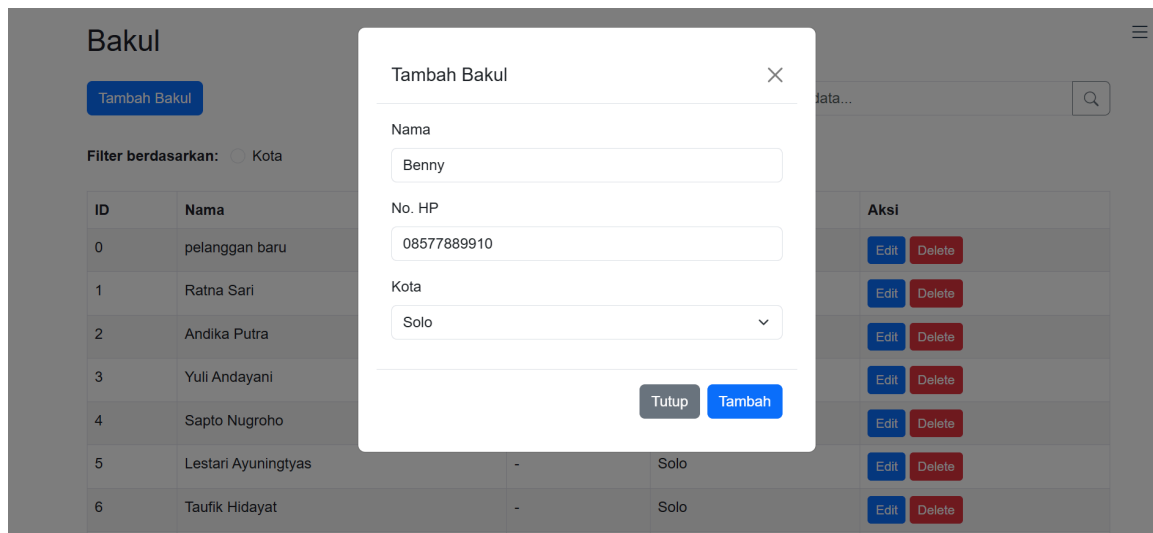
```

id	perusahaan	cp	no_hp	id_kota
18	Abadi Jaya	Ali	081122334455	5

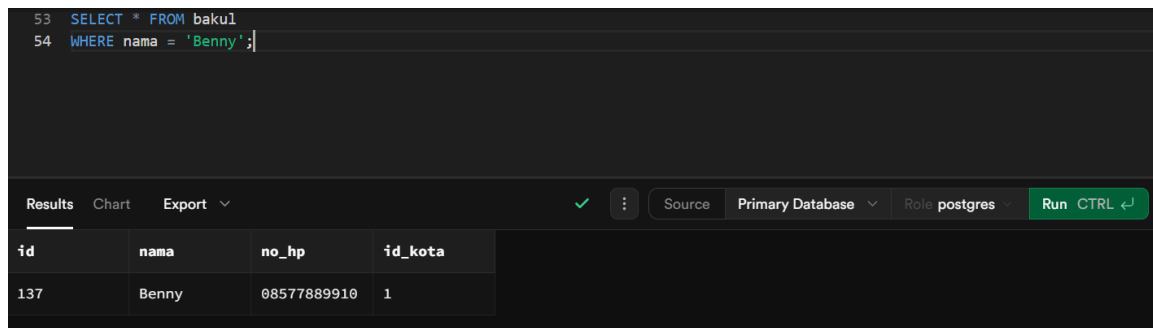
Gambar 4.41 Screenshot hasil data *supplier* pada *database*

4.1.5. Skenario menambahkan data bakul (pelanggan)

Pada skenario ini, ada seorang bakul (pelanggan) yang sebelumnya merupakan pelanggan baru. Kini, bakul tersebut sudah beberapa kali melakukan pembelian di toko dan intensitas pembeliannya meningkat. Oleh karena itu, pemilik toko memutuskan untuk menambahkan data bakul baru. *Input* data bakul baru dapat dilihat pada gambar 4.42, dimana nama bakul adalah Benny dan berasal dari kota Solo. Hasil *query* untuk melihat penambahan data bakul pada *database* dapat dilihat pada gambar 4.43.



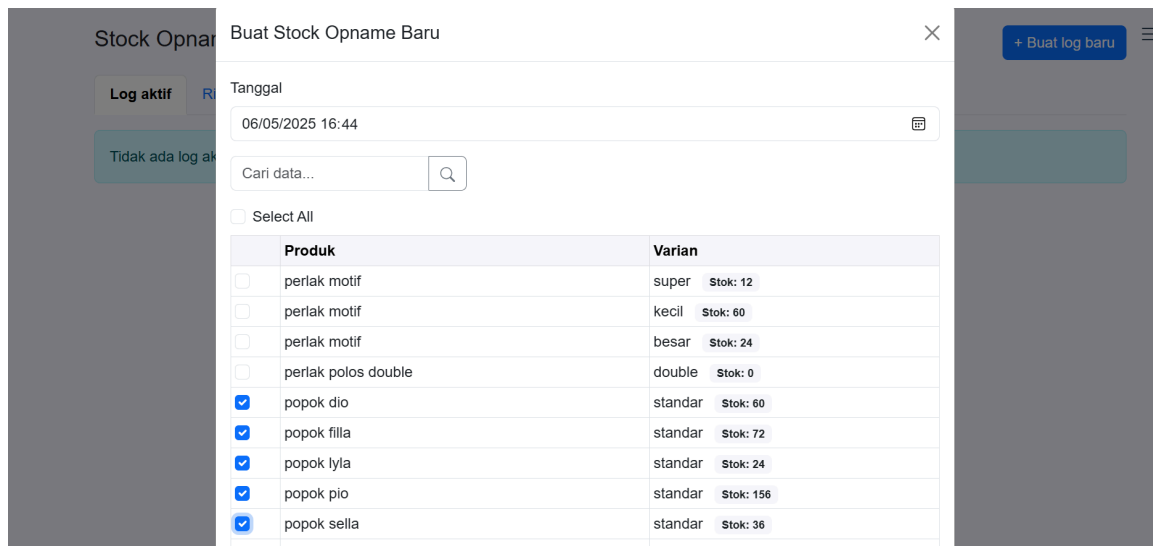
Gambar 4.42 Screenshot tampilan *input* data bakul baru



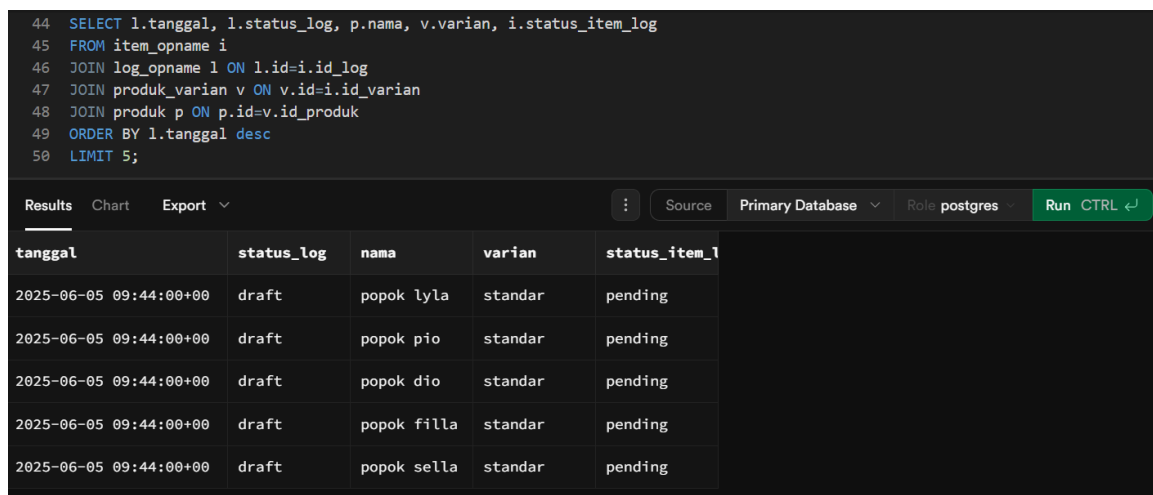
Gambar 4.43 Screenshot hasil data bakul pada *database*

4.1.6. Skenario membuat log *stock opname*

Pada skenario ini, pemilik toko ingin melakukan *stock opname* untuk 5 produk popok yang dimiliki. Kelima produk popok itu ingin diperiksa apakah jumlah stok fisik-nya sesuai dengan data yang ada di sistem. Pemilik toko membuat log baru pada sistem dan memilih kelima produk popok yang ingin diperiksa. Proses pembuatannya dapat dilihat pada gambar 4.44. Sedangkan untuk penambahan data log *stock opname* beserta *item*-nya pada *database* dapat dilihat pada gambar 4.45.



Gambar 4.44 Screenshot tampilan *input* log stock opname baru



Gambar 4.45 Screenshot hasil data log stock opname baru beserta *item*-nya

Saat pemeriksaan fisik dilakukan dan pemilik toko melakukan *input* hasil perhitungan stok fisik pada log, ditemukan bahwa 1 dari 5 produk popok memiliki perhitungan stok yang tidak sama dengan sistem sehingga perlu disesuaikan. Produk bernama “popok sella” memiliki stok fisik sejumlah 48 tetapi data yang tercatat pada sistem adalah 36. Sistem mendeteksi ketidaksesuaian dan menyalakan tombol *match* untuk item tersebut seperti yang terlihat pada gambar 4.46. Pemilik toko menekan tombol tersebut untuk menyesuaikan data stok dan tampilan berubah seperti pada gambar 4.47.

Stock Opname

+ Buat log baru

Log aktif Riwayat

Tanggal: 05/06/2025, 16.44 | Status: draft Edit Hapus Finalisasi

Produk	Varian	Stok Sistem	Stok Fisik	Status	Aksi
popok dio	standar	60	60 (0)	sesuai	Match ✕
popok filla	standar	72	72 (0)	sesuai	Match ✕
popok lyla	standar	24	24 (0)	sesuai	Match ✕
popok pio	standar	156	156 (0)	sesuai	Match ✕
popok sella	standar	36	48 (+12)	tidak_sesuai	Match ✕

Gambar 4.46 Screenshot tampilan jumlah stok tidak sesuai

Stock Opname

+ Buat log baru

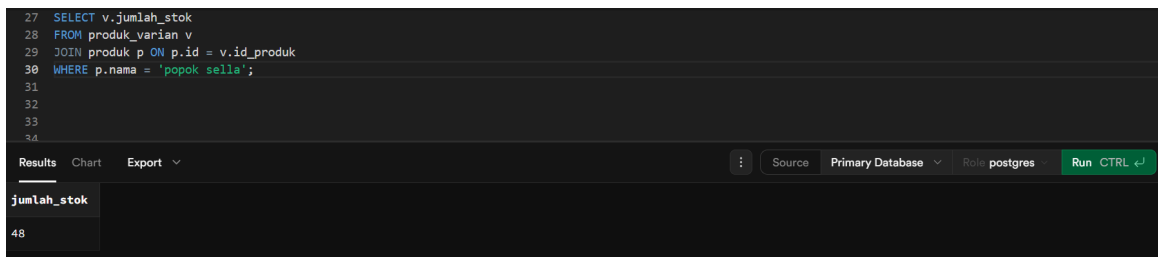
Log aktif Riwayat

Tanggal: 05/06/2025, 16.44 | Status: draft Edit Hapus Finalisasi

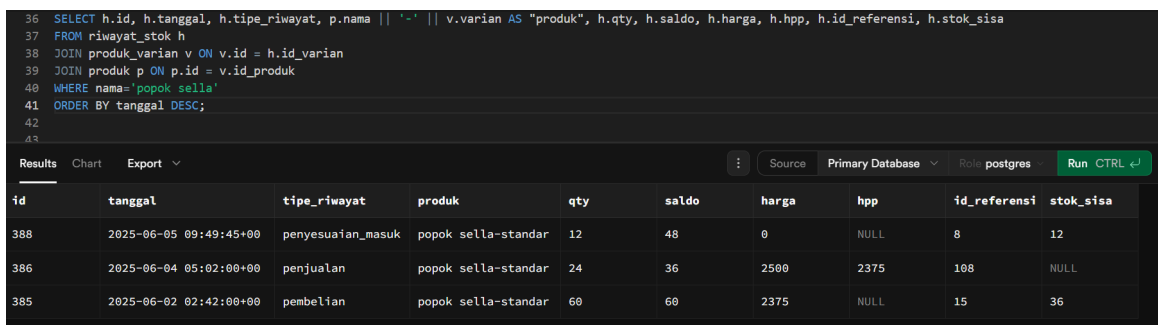
Produk	Varian	Stok Sistem	Stok Fisik	Status	Aksi
popok dio	standar	60	60 (0)	sesuai	Match ✕
popok filla	standar	72	72 (0)	sesuai	Match ✕
popok lyla	standar	24	24 (0)	sesuai	Match ✕
popok pio	standar	156	156 (0)	sesuai	Match ✕
popok sella	standar	48	48 (+12)	disesuaikan	Match ✕

Gambar 4.47 Screenshot tampilan jumlah stok sudah disesuaikan

Perubahan data stok untuk produk tersebut pada *database* dapat dilihat pada gambar 4.48. Jumlah stok kini sudah menyesuaikan stok fisik. Selain itu, hasil penyesuaian ini akan juga akan dicatat sebagai entri baru pada riwayat stok. Pada skenario ini, data penyesuaian akan dikategorikan sebagai tipe penyesuaian masuk dan memiliki “id_referensi” yang sama dengan ID dari log *stock opname*. Hasil *query database* untuk entri riwayat stok ini dapat dilihat pada gambar 4.49.

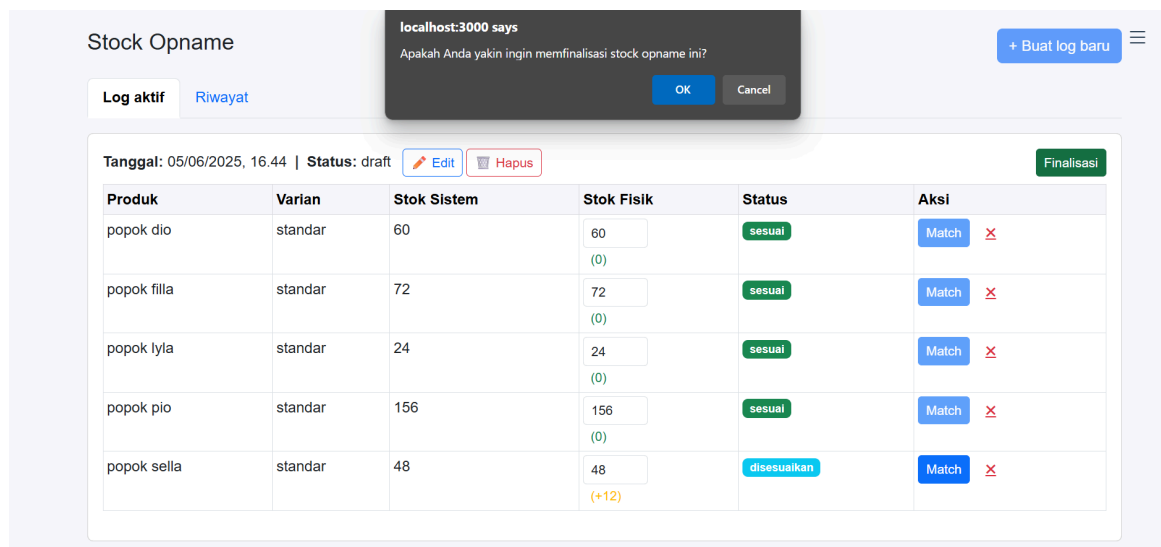


Gambar 4.48 Screenshot hasil data stok setelah penyesuaian



Gambar 4.49 Screenshot hasil data penambahan entri riwayat *stock opname*

Setelah semua item telah disesuaikan, pemilik toko ingin memfinalisasikan *draft* log dengan menekan tombol finalisasi yang kini menjadi aktif. Status log akan berubah menjadi final sehingga tidak dapat diubah lagi. Tampilan *website* dan *database* untuk skenario ini dapat dilihat pada gambar 4.50 dan 4.51.



Gambar 4.50 Screenshot tampilan konfirmasi finalisasi log

```

44 SELECT l.tanggal, l.status_log, p.nama, v.varian, i.status_item_log
45 FROM item_opname i
46 JOIN log_opname l ON l.id=i.id_log
47 JOIN produk_varian v ON v.id=i.id_varian
48 JOIN produk p ON p.id=v.id_produk
49 ORDER BY l.tanggal desc
50 LIMIT 5;

```

tanggal	status_log	nama	varian	status_item_l
2025-06-05 09:44:00+00	final	popok filla	standar	sesuai
2025-06-05 09:44:00+00	final	popok lyla	standar	sesuai
2025-06-05 09:44:00+00	final	popok pio	standar	sesuai
2025-06-05 09:44:00+00	final	popok dio	standar	sesuai
2025-06-05 09:44:00+00	final	popok sella	standar	disesuaikan

Gambar 4.51 Screenshot perubahan data log stock opname final

4.1.7. Skenario membuat laporan

Pada skenario ini, pemilik toko X ingin mengetahui keuntungan yang diperoleh pada kuartal 2 tahun 2025. Pemilik toko X memilih jenis dan periode laporan yang diinginkan kemudian memencet tombol pembuatan. Sistem menampilkan *HTML preview* dari laporan laba rugi yang dibuat. Tampilannya dapat dilihat pada gambar 4.52. Kemudian, pemilik toko X mengunduh laporan dalam bentuk PDF agar dapat dilihat lagi nantinya. Hasil pengunduhan PDF laporan dapat dilihat pada gambar 4.53.

Laporan

Jenis Laporan

Dari Tanggal

Sampai Tanggal

Laporan Laba Rugi

04/01/2025

06/30/2025

Buat Laporan

Unduh PDF

Laporan Laba Rugi 1/4/2025 - 30/6/2025

Keterangan	Jumlah (Rp)
Pendapatan	Rp 74.003.000
Harga Pokok Penjualan (HPP)	Rp 3.475.900

Gambar 4.52 Screenshot tampilan *preview* laporan

LAPORAN LABA RUGI

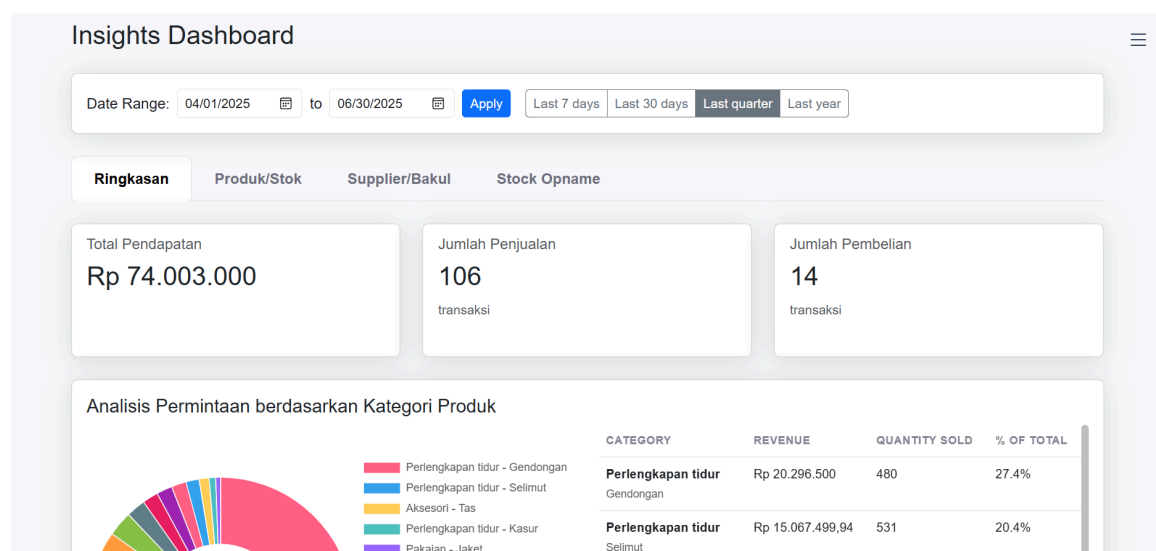
1/4/2025 - 30/6/2025

KETERANGAN	JUMLAH (Rp)
Pendapatan	74.003.000,00
Harga Pokok Penjualan (HPP)	3.475.900,00
Laba Kotor	70.527.100,00
Biaya Operasional	540.000,00
Laba Bersih	69.987.100,00

Gambar 4.53 Screenshot tampilan isi PDF laporan

4.1.8. Skenario melihat *dashboard*

Pada skenario ini, pemilik toko X ingin melihat *insights* apa yang bisa didapatkan dari data pembelian dan penjualan pada kuartal 2 tahun 2025. Pemilik toko memilih *tab* “Ringkasan” dan melakukan *input* rentang tanggal. Tampilan hasilnya dapat dilihat pada gambar 4.54.



Gambar 4.54 Screenshot tampilan dashboard untuk tab “Ringkasan”

4.2. Pengujian berdasarkan survey kepada pemilik

Pengujian ini dilakukan dengan meminta pemilik toko sebagai *user* untuk menjalankan beberapa proses pada sistem. Setelah semua dilakukan, *user* diberikan survei berisi beberapa pertanyaan terkait pengalamannya dalam menggunakan sistem. Pengujian ini ditujukan untuk melihat apakah sistem mudah digunakan oleh *user*. Survei terdiri dari 4 pertanyaan skala *Likert* dan 2 pertanyaan terbuka. Masing-masing hasilnya dapat dilihat pada tabel 4.1 dan 4.2.

Untuk pertanyaan skala *Likert*, skala penilaian memiliki rentang nilai 1 sampai 5, dengan spesifikasi sebagai berikut:

- 1: Sangat tidak setuju
- 2: Tidak setuju
- 3: Netral
- 4: Setuju
- 5: Sangat setuju

Tabel 4.1 Tabel hasil *survey* pertanyaan skala likert

Pernyataan	Skor					Rata-rata skor
	1	2	3	4	5	
Saya dapat memahami alur sistem dengan mudah.	-	-	-	-	2	5
Tampilan sistem dapat membantu saya untuk memahami fungsi yang disediakan.	-	-	-	-	2	5
Saya mampu mendapatkan hasil aksi yang saya inginkan dari sistem.	-	-	-	-	2	5
Saya jarang mengalami kesalahan atau kebingungan saat menggunakan sistem.	-	-	-	1	1	4.5

Dari hasil survei di atas, berikut akan dilakukan perhitungan persentase nilai untuk masing-masing poin pertanyaan. Persentase nilai akan dihitung dengan menggunakan rumus:

$$\text{Persentase Nilai} = \frac{\text{Total jumlah skor} \div 2}{5} \times 100\%$$

*(2) mewakili jumlah responden dan (5) mewakili skor tertinggi.

Pertama, untuk poin kemudahan dalam memahami alur sistem. Responden 1 memberikan skor 5 dan responden 2 juga memberikan skor 5. Maka persentase nilai **kemudahan memahami alur sistem** dapat dihitung seperti berikut:

$$\frac{5}{5} \times 100\% = 100\%$$

Kedua, untuk poin kejelasan tampilan sistem. Kedua responden memberikan skor 5. Maka persentase nilai **kejelasan tampilan sistem** dapat dihitung seperti berikut:

$$\frac{5}{5} \times 100\% = 100\%$$

Ketiga, untuk poin efektivitas sistem. Kedua responden memberikan skor 5. Maka persentase nilai **efektivitas sistem** dapat dihitung seperti berikut:

$$\frac{5}{5} \times 100\% = 100\%$$

Terakhir, untuk poin pencegahan *error*. Responden 1 memberikan skor 4 sedangkan responden 2 memberikan skor 5. Maka persentase nilai **pencegahan error** dapat dihitung seperti berikut:

$$\frac{4.5}{5} \times 100\% = 90\%$$

Tabel 4.2 Tabel hasil *survey* pertanyaan terbuka

Pertanyaan	Jawaban	
	R1	R2
Menurut Anda, manakah bagian sistem yang paling membingungkan?	"Tidak ada"	"Tidak ada"
Apakah saran Anda untuk perbaikan sistem?	"Untuk saat ini belum ada karena masih bisa memenuhi kebutuhan yang ada"	"Belum ada. Saat Ini Tampilan sudah cukup OK"

Kata kunci yang dapat diekstrak dari hasil jawaban kedua responden adalah bahwa saat ini sistem sudah mencukupi kebutuhan dan memiliki tampilan yang baik. Responden juga tidak mengalami kesulitan yang berarti selama menggunakan sistem.

4.3. User Acceptance Testing (UAT)

Pengujian ini dijalankan dengan menggunakan sistem secara langsung pada proses bisnis Toko X. Ada sebanyak 16 skenario yang diuji, mulai dari *login*, pemeriksaan data stok saat ada pesanan, serta pembuatan pembelian dan penjualan. Aksi juga akan mencakup proses *stock opname* yang dilakukan di awal untuk pencatatan data stok awal. Di dalam tiap skenario, akan terdapat beberapa *test case* yang berbeda. Selama pengujian berlangsung, diamati *error* dan kesalahan yang terjadi. Tabel hasilnya dapat dilihat pada tabel 4.3.

Tabel 4.3 Tabel hasil *User Acceptance Testing* (UAT)

No.	Skenario	Test Case	Hasil yang diharapkan	Berhasil /Gagal	Jumlah error
1.	<i>Login</i>	Menguji verifikasi <i>user</i> saat <i>login</i>	Mengautentikasi user dan menampilkan halaman produk saat <i>login</i> berhasil	Berhasil	0
2.	Memeriksa ketersediaan stok	Menguji fitur pencarian produk	Menampilkan produk yang sesuai dengan kata kunci <i>user</i>	Berhasil	0
		Menguji tampilan data produk dan variannya	Menampilkan <i>dropdown</i> tabel varian	Berhasil	0
3.	<i>Create pembelian baru</i>	Menguji penampilan modal <i>form</i>	Menampilkan <i>form</i> pesanan baru saat <i>user</i> menekan tombol	Berhasil	0
		Menguji pemuatan <i>supplier</i>	Menampilkan semua <i>supplier</i> yang ada pada <i>dropdown</i>	Berhasil	0
		Menguji pemuatan produk berdasarkan <i>supplier</i>	Menampilkan <i>field</i> produk saat <i>user</i> menekan tombol. Menampilkan produk yang dimiliki oleh <i>supplier</i> yang dipilih saja. Menonaktifkan <i>field</i> varian sebelum produk dipilih dan setelahnya, menampilkan data varian yang sesuai	Berhasil	0
		Menguji penyimpanan pesanan baru	Menyimpan data ke <i>database</i> dan menampilkan pesan keberhasilan. Membuat kartu pesanan di tab pesanan aktif.	Berhasil	1
4.	<i>Edit status</i>	Menguji	Menonaktifkan <i>toggle</i>	Berhasil	0

	"diterima"	perubahan tampilan <i>form</i>	untuk kembali ke status sebelumnya dan beberapa <i>field</i> lain yang tidak boleh diubah pada status "diterima". Menambahkan <i>fields</i> untuk status "diterima".		
		Menguji perubahan data pesanan	Update <i>database</i> dengan data "diterima". Update tampilan kartu pesanan.	Berhasil	0
5.	Edit status "dibayar"	Menguji perubahan tampilan <i>form</i>	Menonaktifkan <i>toggle</i> untuk kembali ke status sebelumnya beberapa <i>field</i> lain yang tidak boleh diubah pada status "dibayar". Menambahkan <i>fields</i> untuk status "dibayar".	Berhasil	0
		Menguji perubahan data pesanan	Update <i>database</i> dengan data "dibayar". Update tampilan kartu pesanan.	Berhasil	0
6.	Menyelesaikan pesanan	Menguji perubahan dan tampilan pesanan	Mengubah status pesanan. Memindahkan pesanan ke <i>tab</i> riwayat.	Berhasil	0
7.	Create penjualan baru	Menguji penampilan <i>modal form</i>	Menampilkan <i>form</i> pesanan baru saat <i>user</i> menekan tombol	Berhasil	0
		Menguji pemuatan bakul	Menampilkan semua bakul/pelanggan yang ada pada <i>dropdown</i>	Berhasil	0
		Menguji pemuatan	Menampilkan <i>field</i> produk saat <i>user</i>	Berhasil	0

		produk	menekan tombol. Menampilkan semua produk yang dimiliki toko pada <i>dropdown</i>. Menonaktifkan <i>field</i> varian sebelum produk dipilih dan setelahnya, menampilkan data varian yang sesuai		
		Menguji penyimpanan pesanan baru	Menyimpan data ke <i>database</i> dan menampilkan pesan keberhasilan. Membuat kartu pesanan di <i>tab</i> pesanan aktif.	Berhasil	0
8.	Mempersiapkan produk pesanan	Menguji tampilan dan perubahan pesanan	Menampilkan konfirmasi pengubahan status persiapan. <i>Update</i> tampilan kartu pesanan.	Berhasil	0
9.	Edit status "diambil"	Menguji perubahan tampilan form	Menonaktifkan <i>toggle</i> untuk kembali ke status sebelumnya dan beberapa <i>field</i> lain yang tidak boleh diubah pada status "diambil". Menambahkan <i>fields</i> untuk status "diambil".	Berhasil	0
		Menguji perubahan data pesanan	<i>Update database</i> dengan data "diambil" <i>Update</i> tampilan kartu pesanan.	Berhasil	0
10.	Edit status "dibayar"	Menguji perubahan tampilan <i>form</i>	Menonaktifkan <i>toggle</i> untuk kembali ke status sebelumnya beberapa <i>field</i> lain yang tidak boleh	Berhasil	0

			diubah pada status "dibayar". Menambahkan <i>fields</i> untuk status "dibayar".		
		Menguji perubahan data pesanan	<i>Update database</i> dengan data "dibayar" <i>Update</i> tampilan kartu pesanan.	Berhasil	0
11.	<i>Print</i> nota	Menguji tampilan dan hasil <i>print</i> nota	Menampilkan <i>window print</i> atau <i>download</i> PDF. Menampilkan data pembayaran yang sesuai pada nota.	Berhasil	0
11.	Menyelesaikan pesanan	Menguji perubahan dan tampilan pesanan	Mengubah status pesanan. Memindahkan pesanan ke <i>tab</i> riwayat.	Berhasil	0
12.	Membuat log <i>stock opname</i> baru	Menguji penampilan <i>modal</i>	Menampilkan <i>modal form</i> log	Berhasil	0
		Menguji fitur pencarian	Menampilkan produk yang sesuai dengan kata kunci dari <i>user</i>	Berhasil	0
		Menguji pemilihan produk	Mencatat <i>item</i> yang dipilih <i>user</i> dengan benar. Menandai <i>item</i> yang dipilih oleh <i>user</i> .	Berhasil	0
		Menguji penyimpanan log baru beserta <i>item</i> -nya.	Menyimpan data ke <i>database</i> dan menampilkan pesan keberhasilan. Membuat log pada <i>tab</i> log aktif. Menonaktifkan tombol pembuatan log.	Berhasil	1

13.	Input data log	Menguji perbandingan data stok dan tampilannya	Mengubah status berdasarkan hasil perbandingan. Mengaktifkan tombol <i>match</i> untuk data yang tidak sesuai	Berhasil	0
14.	Menyesuaikan data stok yang tidak sesuai	Menguji perubahan data dan tampilan	Mengubah data stok dan menambahkan riwayat stok. Menampilkan pesan keberhasilan.	Berhasil	0
15.	Memfinalisasikan log	Menguji perubahan dan tampilan log	Mengubah status log dan <i>item</i> -nya. Memindahkan log ke <i>tab</i> riwayat. Mengaktifkan tombol pembuatan log.	Berhasil	0

Ada 30 *test case* yang diuji dan hasilnya menunjukkan adanya 2 *error*. Tetapi kedua *error* ini tidak fatal dan dapat diklasifikasikan sebagai kesalahan minor. *Error* yang pertama terdapat pada modul pembelian, dimana terkadang penyimpanan data pesanan dapat terjadi lebih dari 1x. Hal ini terjadi karena *event listener* yang dilampirkan pada tombol penyimpanan tidak dibersihkan sehingga menjadi menumpuk pada kasus tertentu. *Error* kedua adalah terkait konversi tanggal pada modul *stock opname*. Seharusnya *database* menyimpan data tanggal dengan *timezone UTC*. Tetapi pada modul *stock opname*, tanggal disimpan dengan *timezone* lokal sehingga menyebabkan hasil konversi pada tampilan menjadi salah dan tidak konsisten. Jika dilakukan perhitungan terhadap tingkat *error* yang terjadi menggunakan rumus berikut:

$$Error (\%) = \frac{Total\ error}{Total\ test\ case} \times 100\%$$

Maka, akan didapatkan bahwa tingkat *error* pada sistem saat UAT adalah sekitar 6.67%.

4.4. Pengujian efisiensi sistem

Pengujian ini, secara inti, ingin membandingkan serta mengukur beberapa metrik kuantitatif terkait efisiensi operasional, khususnya pada manajemen stok. Untuk pengujian perbandingan, yang diuji adalah efisiensi proses saat menggunakan sistem dan tidak

menggunakan sistem. Tujuannya adalah untuk melihat apakah sistem mampu meningkatkan efisiensi kegiatan operasional.

4.4.1. Waktu pengecekan ketersediaan stok

Tanpa menggunakan sistem, proses yang berlangsung dalam mengecek ketersediaan stok adalah sebagai berikut, pemilik toko akan mengira-ngira ketersediaan stok ketika pesanan masuk dan menerima pesanan. Kemudian, pemilik akan mengecek secara fisik ketersediaan stok. Sedangkan dengan penggunaan sistem, pemilik dapat mencari produk yang diinginkan dan melihat data jumlah stok. Kedua proses akan dijalankan secara independen dan diukur waktunya dengan menggunakan *stopwatch*. Proses pengujian tanpa sistem dianggap hanya mencakup waktu penerimaan pesanan dan pengecekan stok sehingga tidak termasuk waktu tambahan seperti mencari barang, dll. Pengujian akan dilakukan dengan 3 variabel jumlah produk yang berbeda dan setiap variabel akan diuji sebanyak 5x percobaan. Pertama, pengujian akan dilakukan untuk pemeriksaan 1 barang. Kedua, pengujian untuk pemeriksaan 3 barang. Ketiga, pengujian untuk pemeriksaan 5 barang. Perhitungan efisiensi waktu akan menggunakan rumus di bawah ini:

$$Efisiensi = \frac{Waktu\ tanpa\ sistem - Waktu\ dengan\ sistem}{Waktu\ tanpa\ sistem} \times 100\%$$

Hasil pengukuran dan perhitungan efisiensi dari ketiga variabel dapat dilihat pada tabel 4.4. Tabel akan menampilkan rata-rata waktu yang didapat dari **5 percobaan** untuk masing-masing variabel.

Tabel 4.4 Tabel hasil perbandingan waktu pengecekan stok

Variabel jumlah produk	Rata-rata Waktu (Detik)		Tingkat efisiensi (%)
	Tanpa sistem (5x)	Dengan sistem (5x)	
1 produk	19.07	8.72	54.27
3 produk	53.35	23.38	56.18
5 produk	76.05	32.98	56.63
Rata-rata			55.69

Hasil pengukuran menunjukkan bahwa proses pengecekan ketersediaan stok **tanpa** menggunakan sistem, pada ketiga variabel, memakan waktu kurang lebih **2x lebih lama** dibandingkan dengan proses yang menggunakan sistem. Terutama dengan bertambahnya jumlah produk yang harus diperiksa, efisiensi meningkat apabila menggunakan sistem. Hasil perhitungan tingkat efisiensi menunjukkan bahwa sistem mampu meningkatkan efisiensi pengecekan ketersediaan stok sebesar **54.27%** untuk pemeriksaan 1 produk, **56.18%** untuk pemeriksaan 3

produk, dan **56.63%** untuk pemeriksaan 5 produk. Secara rata-rata, sistem mampu meningkatkan efisiensi pemeriksaan stok sebesar **55.69%**. Hasil ini artinya, bahwa dari segi waktu pengecekan ketersediaan stok, sistem mampu meningkatkan efisiensi kegiatan.

4.4.2. Tingkat kesalahan perhitungan stok

Untuk pengukuran metrik ini, akan diambil **8 varian produk** sebagai sampel. Sampel produk dipilih berdasarkan beberapa kategori untuk memberikan variasi dalam sampel. Terdapat 4 kategori pemilihan produk: pertama, berdasarkan kecepatan pergerakan barang. Diambil 1 produk yang pergerakannya cepat dan 1 produk yang pergerakannya lambat. Kedua, berdasarkan harga barang. Diambil 1 barang yang harganya tinggi dan 1 barang yang harganya rendah. Ketiga, berdasarkan penempatan barang di toko. Diambil 1 produk yang dekat dengan kasir dan 1 yang penempatannya lebih jauh. Terakhir, berdasarkan kuantitas minimum penjualan barang. Semua produk di toko X dijual lusinan tetapi ada produk yang boleh dibeli di bawah 12 buah, seperti 3 buah ($\frac{1}{4}$ lusin) atau 6 buah ($\frac{1}{2}$ lusin). Diambil 1 produk yang kuantitas minimum-nya 1 lusin dan 1 produk yang kuantitas minimum-nya di bawah 1 lusin.

Kedelapan varian tersebut akan dicatat stok awal-nya pada hari pertama penggunaan sistem. Selama masa pengujian sistem, 2 metode pencatatan stok (tanpa dan dengan sistem), akan dijalankan secara bersamaan. Kemudian, pada akhir masa pengujian, dicatat kembali stok akhir dari 8 varian produk tersebut dan dibandingkan hasilnya antara kedua metode. Stok akhir dari metode tanpa sistem akan didapatkan dari rekoleksi pemilik toko X. Sedangkan stok akhir dari metode dengan sistem akan dilihat dari sistem secara langsung. Tujuannya adalah untuk membandingkan tingkat kesalahan perhitungan stok di antara kedua metode. Jumlah stok secara fisik yang dihitung pada akhir masa pengujian untuk masing-masing produk sampel akan menjadi nilai patokan kebenaran. Tabel perbandingan dapat dilihat pada tabel 4.5.

Tabel 4.5 Tabel hasil perbandingan tingkat kesalahan data stok

Varian produk	Stok awal	Stok Akhir		
		Fisik	Tanpa sistem	Dengan sistem
Pergerakan barang				
Seporet (Singlet bayi) - Ukuran M	60	0	0	0
Sofa bayi (j) - Kecil	48	42	42	42
Harga barang				

Gendongan depan hipsit - Boneka	84	24	78	78
Popok lyla	108	78	24	24
Penempatan barang				
Celana kacamata (I) - Bordir	60	0	0	0
Selimut Clara - Kecil	72	225	220	225
Kuantitas minimum				
Grito bayi - Perekat	72	6	0	6
Kasur - Kotak	60	6	6	6

Hasil pengujian menunjukkan bahwa **sistem** berhasil mencatat data stok dengan akurat, dimana terdapat **0 kesalahan** dari 8 sampel produk yang diuji. Sedangkan, manajemen stok **tanpa** menggunakan **sistem** menunjukkan adanya **2 kesalahan** data stok. Untuk kesalahan pertama, yaitu pada produk “selimut clara”, pemilik toko X tidak dapat menyebutkan angka pasti dari stok akhir karena volume yang cukup besar dan riwayat yang cukup banyak. Kemudian, untuk kesalahan kedua pada produk “grito bayi” varian perekat, terjadi kebingungan karena ada seorang pembeli yang membeli sejumlah $\frac{1}{2}$ lusin saja padahal kuantitas minimum pembelian produk “grito bayi” adalah 1 lusin. Hal ini dapat terjadi karena pembeli mencampur dengan varian lain dari produk yang sama. Variasi data historis seperti ini dapat dicatat oleh sistem dan data stok dihitung secara otomatis sehingga mengurangi kemungkinan adanya *human-error*. Jika dihitung dari hasil pengujian, tingkat kesalahan tanpa sistem adalah 25% sehingga dapat diartikan bahwa sistem mampu **mengurangi kesalahan** perhitungan stok sebesar **25%**.

4.4.3. Waktu pengumpulan data dan pembuatan laporan

Dengan sistem bisnis yang saat ini berlangsung, dimana tidak ada pencatatan khusus untuk manajemen stok, proses pengumpulan data dan penyusunan laporan akan memakan waktu cukup lama. Hal ini dikarenakan data pembelian dan penjualan perlu dikumpulkan dari nota-nota yang ada. Begitu juga dengan data lainnya, seperti data *supplier* dan bakul, yang membutuhkan waktu dalam proses rekoleksinya. Sedangkan dengan menggunakan sistem, data terkait manajemen stok telah terintegrasi dan *user* dapat membuat laporan dengan mudah dan cepat. Pengujian ini akan berfokus untuk mengukur waktu yang dibutuhkan dan performa sistem dalam mengumpulkan data dan membuat laporan. Tujuannya adalah untuk melihat seberapa efektif penggunaan sistem dalam aspek ini. Pertama, dilakukan 5x percobaan berbeda untuk mendapatkan rata-rata waktu yang dibutuhkan *user* dalam membuat laporan dengan sistem.

Tabel percobaan dapat dilihat pada tabel 4.6. Untuk memastikan konsistensi dalam percobaan, kelimanya akan menggunakan jenis dan periode laporan yang sama. Waktu diukur mulai dari *user* memilih jenis dan periode hingga *preview* laporan ditampilkan.

Tabel 4.6 Tabel pengukuran waktu pembuatan laporan dengan sistem

Percobaan	Waktu (Detik)
1	9.69
2	8.49
3	8.74
4	8.14
5	8.83
Rata-rata	8.78

Hasil menunjukkan bahwa secara rata-rata, hanya dibutuhkan **8.78 detik** untuk membuat laporan dengan menggunakan sistem. Kemudian, pengujian yang kedua akan membandingkan performa sistem dalam membuat laporan berdasarkan jumlah datanya. Kelompok percobaan 1 akan menggunakan 50% data dari seluruh data yang berhasil dikumpulkan selama masa pengujian pada sistem sedangkan kelompok percobaan 2 akan menggunakan seluruh data yang ada. Hasil percobaan dapat dilihat pada tabel 4.7. Pengujian ini berfokus pada waktu yang dibutuhkan sistem dalam menghasilkan laporan sehingga hanya diukur mulai dari tombol pembuatan ditekan hingga *preview* laporan dihasilkan.

Tabel 4.7 Tabel perbandingan performa pembuatan laporan berdasarkan jumlah data

Percobaan	Waktu (Detik)	
	50% volume data	100% volume data
1	0.83	1.00
2	1.03	1.28
3	0.89	0.94
4	0.84	1.28
5	0.86	0.89
Rata-rata	0.89	1.08

Dapat dilihat dari hasil percobaan, bahwa rata-rata waktu yang dibutuhkan untuk 50% volume data lebih cepat ketimbang dengan waktu yang dibutuhkan untuk 100% volume data. Secara rata-rata, sistem membutuhkan waktu **0.89 detik** untuk menghasilkan laporan dengan 50% volume data dan **1.08 detik** dengan 100% volume data sehingga dapat dikatakan bahwa performa pembuatan laporan dipengaruhi oleh jumlah data. Meskipun begitu, tidak ada perbedaan waktu yang signifikan untuk menandakan kelambatan dalam pembuatan laporan. Tetapi, perlu diingat juga bahwa data sampel yang dimiliki tidak dalam jumlah yang begitu besar dan kecepatan internet juga mempengaruhi proses pengambilan data karena sistem menggunakan *cloud database* milik *Supabase*.